

Pathashilpa

Bhashini API Use Case · Smart India Hackathon 2026
AI-Driven Market Linkage and Smart Cataloging for Marginalized Artisans

1. The problem we are solving

A marketplace listing requires four things: a clean photograph, product copy in English, a defensible price, and completed registration. Rural artisans can produce none of them. Every existing platform begins *after* those exist. The artisan reaches "Create Listing" and stops.

Figure	Meaning	Source
0.2%	share of handloom sales occurring online	4th All India Handloom Census
95.5%	rural mobile owners who own a smartphone	NSO Telecom Survey, 2025
67%	handloom households earning under ₹5,000/month	PIB, Ministry of Textiles
35.2 lakh	handloom weavers and allied workers	PIB, Ministry of Textiles

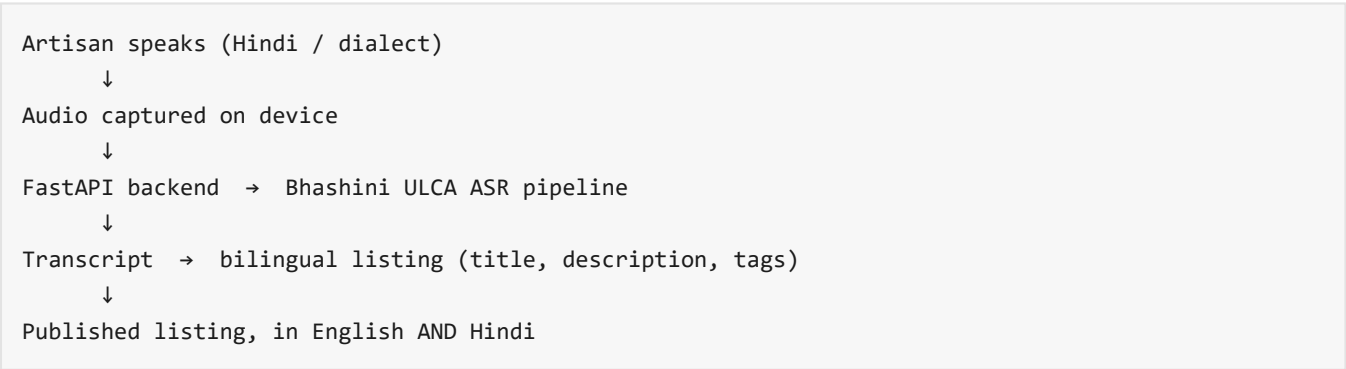
The insight: 0.2% online against 95.5% smartphone ownership. The device barrier is already gone. What is missing is software the artisan can operate — and the single largest obstacle is the requirement to write in English. **This is precisely the gap Bhashini closes for us.**

2. How Bhashini is used

Our target user is an artisan such as Kamala, 42, a Chanderi weaver who uses WhatsApp voice notes daily but does not type, reads Hindi slowly, and speaks no English. She should never have to write anything.

2.1 ASR — Automatic Speech Recognition (*primary, implemented*)

The artisan photographs the craft, then simply **speaks about it** in Hindi or a regional dialect. Bhashini's ASR converts that speech to text, and that transcript becomes the raw material for a complete bilingual product listing.



We call the standard two-step ULCA flow: `getModelPipeline` to obtain the pipeline configuration and callback endpoint, then the compute call with base64 `audioContent` and the source language. The pipeline configuration is cached per language for the session rather than re-fetched on every request.

2.2 TTS — Text-to-Speech (*planned*)

Our pricing engine calculates a fair wage-based price and explains its reasoning. For an artisan with limited literacy, **hearing** that explanation in their own language is what builds trust in the number. We intend to use Bhashini TTS to read the suggested price and the generated description aloud before the artisan confirms.

2.3 Supporting services

Service	Purpose in Pathashilpa
Denoiser	Artisans record inside working workshops — loom clatter, wheels, open courtyards. Denoising directly improves ASR accuracy in our real environment.
VAD	Trims silence so recordings stay within our 100 KB per-clip sync budget on 2G/3G connections.
Punctuation	Raw transcripts carry none; punctuation materially improves the quality of the generated listing copy.
NMT	Cross-checking Hindi↔English listing copy where required.
OCR	Reserved for artisan identity-document verification in a later phase.

3. Languages

The application ships complete interfaces in **Hindi, English and Bengali**, with every artisan-facing screen fully translated. Speech language is deliberately *independent* of interface language — an artisan may run the interface in Hindi while speaking Bundeli or another regional dialect. Our roadmap targets the full set of scheduled languages Bhashini supports, which is a primary reason for choosing Bhashini over a single-language commercial provider.

4. Architecture and key handling

Bhashini credentials are never shipped in the mobile application. All Bhashini calls are made server-side from our FastAPI backend, which holds the `userID` and `ulcaApiKey` as server environment variables. The mobile app authenticates to our backend with a verified Firebase ID token, and the backend proxies the request. This prevents key extraction from the APK and protects the quota from abuse.

Layer	Technology	Role
Mobile client	Flutter (Android 8.0+)	Capture, offline drafts, UI — holds no API keys
Backend	FastAPI (Python 3.11)	Bhashini calls, key custody, per-user rate limiting
Identity	Firebase Auth	Phone OTP; backend verifies every token

Speech recognition degrades gracefully across three tiers so the artisan is never blocked: cloud ASR via Bhashini when connected, the on-device recogniser when the network is weak, and a guided form as a final fallback. Requests are rate limited per authenticated user to protect the shared public quota.

5. Expected usage

Metric	Estimate
--------	----------

Audio length per request	10–30 seconds
ASR calls per listing	1 — one spoken description per product
Hackathon / pilot volume	Low hundreds of calls during development and demonstration
Rate limiting	Enforced server-side, per authenticated user

Usage is inherently modest: one ASR call is made per *product listed*, not per view or per session. A listing takes an artisan roughly ninety seconds to create, which naturally bounds request volume.

6. Why Bhashini

- **Indic language depth.** Our users speak Hindi and regional dialects, not English. Bhashini is built for exactly this.
 - **Public digital infrastructure.** As free national infrastructure, Bhashini keeps our operating cost near zero — which is what lets us commit that the artisan never pays and gives up no margin.
 - **Alignment of purpose.** Bhashini exists to remove language as a barrier to digital participation. That is, precisely, the barrier keeping 35.2 lakh weavers off the internet.
-

Pathashilpa — "Your craft. Your price. Your name."

Submitted for Smart India Hackathon 2026 · Application name: `pathashilpo`

Platform: Android mobile application (Flutter) with a FastAPI backend.